UT San Antonio
EE 4463, EE 5573, CS 6243
Spring 2026
Assignment: HW-3

1 Assignment Instructions

- **Submission.** Submit a single PDF on Canvas by the deadline.
- **Deliverables.**
  - Complete Python code covering all tasks, with detailed comments.
  - A report that includes all requested visualizations (figures with captions), analyses, and discussions.
- **References and Originality.** Any external source (notes, books, online resources) must be cited in a *Reference List*. Do *not* outsource any part of the assignment to another (naturally or artificially) intelligent entity. Except for explicitly cited content, submission of the assignment attests that the work is your own.
- **Grading.** Three equally weighted criteria: **Correctness**, **Completeness**, and **Clarity**.

  There are 12 problems in this assignment (7 in Part I and 5 in Part II). Points are allocated equally within each part:
  - Part I (Problems 1–7): 10 points each (70 points total)
  - Part II (Problems 1–5): 6 points each (30 points total)
- **Teamwork.**
  - Work in the assigned team (see Canvas).
  - Each student must submit the same team report individually on Canvas before the deadline to receive a grade.
  - No extensions or late submissions.
  - Individual grades may differ for imbalanced effort.

2 Part I: Parametric and Non-Parametric Regression

2.1 Datasets

This assignment uses two datasets. Follow the specifications below for loading, splitting, and preprocessing. All random operations must use `numpy.random.seed(42)`.

2.1.1 Dataset A: California Housing

- **Source:** `sklearn.datasets.fetch_california_housing`
- **Samples:** $N = 20640$
- **Feature:** `MedInc` (column 0) $\rightarrow$ scalar $x \in \mathbb{R}^N$
- **Target:** median house value ($y \in \mathbb{R}^N$, in \$100k)
- **Split:** Perform *one* global shuffle of indices. Use:
  - Training: indices $[0 : \texttt{train_size}]$ (max $\texttt{train_size} = 10000$)
  - Fixed test: indices $[10000 : 15000]$ (size 5000)
  - Ignore remaining samples
- **Standardization:** Features only (training mean/std). Do *not* standardize y for MSE.

2.1.2 Dataset B: Auto-MPG

- **Source:** `https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data`
- **Format:** Whitespace-delimited. Column names: `mpg, cylinders, displacement, horsepower, weight, acceleration, model_year, origin, car_name`
- **Preprocessing:** Drop rows with missing `horsepower`
- **Features (7):**
  - x_1: `cylinders`, x_2: `displacement`, x_3: `horsepower`
  - x_4: `weight`, x_5: `acceleration`
  - x_6: `model_year`, x_7: `origin`
- **Target:** `mpg` (y)
- **Split:** Global permutation with `numpy.random.seed(42)`. Use:
  - Training: first $n_{\text{train}} = 300$ indices
  - Test: next 92 indices (fixed)
- **Standardization:** Feature rows only (training mean/std). Do *not* standardize y for MSE.

2.2 Conventions

ii. **Dataset and Split (California Housing).** Use `sklearn.datasets.fetch_california_housing`. There are $N = 20640$ samples. Perform *one* global shuffle of all indices with `numpy.random.seed(42)` and `numpy.random.permutation`. The largest training size used in this HW is 10,000. Use indices [0:10000] for training and [10000:15000] for a fixed test set of size 5,000. For any smaller training size $N \leq 10{,}000$, use [0:N] as training while keeping the *same* fixed test set [10000:15000]. Ignore all remaining samples.

ii. **Standardization policy.** Features only: standardize using *training* statistics (mean and standard deviation computed on the current training set) and apply the same transform to the test features. **Do not standardize the target** y for reporting; all MSE must be in original y-units. If you temporarily standardize y for any internal step, inverse-transform the predictions before computing MSE.

Z-score normalization (applied to every feature) is defined as

$$z_i = \frac{x_i - \mu}{\sigma}, \quad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}.$$

ii. **MSE definition.** For a test set $\{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$,

$$\text{MSE} = \frac{1}{n_{\text{test}}}\sum_{i=1}^{n_{\text{test}}}(\hat{y}_i - y_i)^2.$$

ii. **Environment and tools.** Python $\geq$ 3.9, NumPy, Matplotlib. Set `numpy.random.seed(42)` at the start of each run. **Do not use scikit-learn** for the core algorithms (allowed only to load the dataset).

ii. **Efficiency in 1D.** For radius-based and KNN regression on a scalar feature, sort x_{train} once and use sliding-window / two-pointer methods to avoid $O(n_{\text{train}}\, n_{\text{test}})$ brute force. Excessive inefficiency will be penalized.

2.3 Problems

2.3.1 Problem 1

Load the California Housing Dataset. Extract `MedInc` (column 0) as the scalar feature $x \in \mathbb{R}^N$ and the median house value as the target $y \in \mathbb{R}^N$. Use the Conventions for splitting and standardization. Perform the following tasks.

i. **Compute** `split_data(x, y, train_size)`: using the single global permutation (seed 42), return $(x_{\text{train}}, x_{\text{test}}, y_{\text{train}}, y_{\text{test}})$ with training indices [0 : `train_size`] and the fixed test indices [10000 : 15000]. Raise `ValueError` if `train_size` > 10000.

i. **Compute** Verify sizes for `train_size` $\in \{1000, 5000, 10000\}$.

i. **Compute** Implement `standardize(v)` that performs the z-score normalization defined in the Conventions section. Apply standardization to features only: compute μ_x, σ_x on x_{train} and apply the same transformation to x_{test}. Do *not* standardize y for MSE reporting.

i. **Plot Fig. 0**: side-by-side scatter plots for the training subset with `train_size` $= 5000$: (left) unstandardized $(x_{\text{train}}, y_{\text{train}})$; (right) standardized x_{train} vs y_{train}. Label axes (include units: MedInc and median house value are in \$100k).

2.3.2 Problem 2

Implement radius-based regression from scratch using NumPy. Operate on standardized features (per the Conventions section). For each test point $x_i \in x_{\text{test}}$, predict $\hat{y}_i$ as the mean of y_j for all training points satisfying $|x_j - x_i| \leq C$. If no training point lies within radius C, use the mean of y_{train} as an explicit fallback. Evaluate using test MSE in the original y-units. Perform the following tasks.

ii. **Compute** `radius_predict(x_train, y_train, x_test, C)`.

ii. **Compute** Fix `train_size` $= 5000$. For $C \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.3, 0.5, 1, 2, 3, 4, 5\}$ (standardized x-scale), compute the test MSE.

ii. **Plot Fig. 1**: test MSE (y-axis) vs C (x-axis).

ii. **Compute** Fix $C = 0.1$. For `train_size` $\in \{1000, 2000, \ldots, 10000\}$, compute test MSE using nested training prefixes and the fixed test set.

ii. **Plot Fig. 2**: test MSE (y-axis) vs training size (x-axis).

ii. **Discuss** Figs. 1–2. Explain the observed trends with respect to C and training size. Report the fraction of test points that used the fallback for each C, and discuss its impact.

2.3.3 Problem 3

Implement K-nearest neighbors regression from scratch using NumPy. Operate on standardized features (per the Conventions section). For each test point x_i, predict $\hat{y}_i$ as the mean of y_j over the K nearest neighbors in x_{train} (Euclidean distance). If $K > n_{\text{train}}$, use all training points. Evaluate using test MSE. Perform the following tasks.

iii. **Compute** `knn_predict(x_train, y_train, x_test, K)`.

iii. **Compute** Fix `train_size` $= 5000$. For $K \in \{1, 3, 5, 10, 20, 50, 100, 500, 1000\}$, compute the test MSE.

iii. **Plot Fig. 3**: test MSE (y-axis) vs K (x-axis).

iii. **Compute** Fix $K = 100$. For `train_size` $\in \{1000, 2000, \ldots, 10000\}$, compute test MSE using the nested splits.

iii. **Plot Fig. 4**: test MSE (y-axis) vs training size (x-axis).

iii. **Discuss** Figs. 3–4 and compare the results to radius-based regression.

2.3.4 Problem 4

Implement univariate polynomial regression where x is the single scalar feature (MedInc). Build the design matrix $X_{\text{poly}} \in \mathbb{R}^{(d+1)\times n}$ with columns $[1, x_i, x_i^2, \ldots, x_i^d]^\top$. Train the weight vector $w \in \mathbb{R}^{d+1}$ by solving the least-squares problem via SVD. Follow the standardization policy defined in the Conventions section, with the additional requirement that each non-bias row of X_{poly} (i.e., each power column $k = 1, \ldots, d$) must be row-standardized using training-set row statistics.

Perform the following tasks.

iv. **Compute** `poly_features(x, degree)`: given a column vector x (already standardized by μ_x, σ_x), return X_{poly} with columns $[1, x_i, \ldots, x_i^{\text{degree}}]^\top$. Then apply row-standardization to the non-bias rows using the training statistics.

iv. **Compute** `solve_LS(X_poly, y)`: SVD-based least-squares solution for w.

iv. **Plot Fig. 5**: scatter plot of standardized x_{train} vs y_{train} for `train_size` $= 5000$. Label axes (include units for y).

iv. **Compute** With `train_size` $= 5000$, for $d \in \{1, 2, 3, 4, 5, 10, 15, 20\}$, train on $(x_{\text{train}}, y_{\text{train}})$ and report both train and test MSE.

iv. **Plot Fig. 6**: train (blue) and test (red) MSE vs d.

iv. **Compute** Fix $d = 9$. For `train_size` $\in \{1000, 3000, 5000, 7000, 10000\}$, compute train and test MSE using the nested split. Recompute all training statistics $(\mu_x, \sigma_x, \mu_k, \sigma_k)$ for each training size and apply the same transform to the test set.

iv. **Plot Fig. 7**: test MSE (y-axis) vs training size (x-axis) for $d = 9$.

iv. **Plot Fig. 8**: for $d = 9$ and `train_size` $= 5000$, show the scatter of x_{train} vs y_{train} (blue dots) and overlay the fitted curve $(x_{\text{train}}, X_{\text{poly}}^\top w)$ as a red line. Sort by x_{train} for a smooth curve.

iv. **Discuss** Figs. 5–8: describe the shape of the data; explain the MSE trends with respect to d and training size; identify underfitting or overfitting; evaluate the quality of the $d = 9$ fit.

2.3.5 Problem 5

The dataset is available online at `https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data`. Load the whitespace-delimited CSV with column names `mpg, cylinders, displacement, horsepower, weight, acceleration, model_year, origin, car_name` and drop rows with missing `horsepower`. Use all seven numeric features: cylinders (x_1), displacement (x_2), horsepower (x_3), weight (x_4), acceleration (x_5), model year (x_6), and origin (x_7). Define the matrices TRI $\in \mathbb{R}^{7\times n_{\text{train}}}$, TRO $\in \mathbb{R}^{n_{\text{train}}\times 1}$, TEI $\in \mathbb{R}^{7\times n_{\text{test}}}$, and TEO $\in \mathbb{R}^{n_{\text{test}}\times 1}$ using a global permutation with `numpy.random.seed(42)`, $n_{\text{train}} = 300$, and a fixed test set of size 92. Perform the following tasks.

v. **Compute** Standardize each feature row of TRI using the z-score normalization defined in the Conventions section and apply the same transformation to TEI. Do *not* standardize TRO or TEO for MSE reporting.

v. **Compute** Compute the 8×8 absolute Pearson correlation matrix C among $\{x_1, \ldots, x_7, y = \text{mpg}\}$. Present C. Which input correlates most strongly with y?

v. **Plot Fig. 9**: TRO vs TRI$[0, :]$ (cylinders). **Fig. 10**: TRO vs TRI$[3, :]$ (weight). Use appropriate labels and titles. Create one additional figure showing all seven features versus mpg in a 3×3 grid (apply jitter to the discrete features: cylinders, model year, origin).

2.3.6 Problem 6

For each of the seven features separately, fit univariate polynomial regression with degrees $M \in \{0, 1, 2, 3, 4, 5\}$ using the same SVD-based least-squares framework and the standardization policy defined in the Conventions section. Perform the following tasks.

vi. **Plot Fig. 11** (weight, $M = 3$): predicted curve over a dense standardized input grid (raw domain $[1613, 5140]$) together with the TRO vs TRI$[3, :]$ scatter. Include labels, legend, and title.

vi. **Plot Fig. 12** (horsepower, $M = 3$): predicted curve over a dense standardized input grid (raw domain $[46, 230]$) together with the TRO vs TRI$[2, :]$ scatter.

vi. **Plot Fig. 13**: test MSE versus M for all seven features (different colors). Which input is most predictive? Relate your answer to the correlations computed in Problem 5.

2.3.7 Problem 7

Extend to multivariate polynomial regression of total degree $M \in \{1, 2, 3\}$ using the standardized features (x_1 = cylinders, x_2 = displacement, x_3 = horsepower, x_4 = weight, x_5 = acceleration, x_6 = model year, x_7 = origin) and the SVD-based least-squares framework. Include all monomials of total degree $\leq M$. Perform the following tasks.

vii. **Present** For each $M \in \{1, 2, 3\}$, write the explicit model terms (e.g., for $M = 2$: $1, x_1, \ldots, x_1^2, x_1 x_2, \ldots$).

vii. **Compute** Train on TRI/TRO and evaluate on both training and test sets for each M. Report train and test MSE.

vii. **Plot Fig. 14**: MSE (y-axis) vs M (x-axis), with training (red) and testing (blue) curves. Compare the best multivariate test MSE to the best univariate result from Problem 6. Discuss complexity, overfitting, and generalization.

3 Part II: Classification, SGD, Cross-Validation

3.1 Dataset

You will work with the Wine dataset, a classic benchmark for classification. It contains 178 samples with 13 features and 3 classes. The dataset is available at `https://archive.ics.uci.edu/dataset/109/wine`. All random operations in Part B must use `numpy.random.seed(42)` for reproducibility.

3.2 Problems

3.2.1 Problem 1

Perform the following tasks.

i. Load the Wine dataset and extract the features matrix $\mathbf{X}$ and the label vector $\mathbf{y}$.

i. Filter the dataset to retain only the first 40 samples from each class.

i. Perform feature scaling using the z-score normalization defined in the Conventions section (population mean and standard deviation with divisor n). Apply the transformation to all features.

3.2.2 Problem 2

Perform the following tasks.

ii. **Implement** the `softmax` function that takes a vector of logits and returns the softmax probabilities. Subtract the maximum logit value before exponentiation to ensure numerical stability.

ii. **Implement** the `kfold_split` function for K-fold cross-validation.

ii. **Implement** the `accuracy` function that manually computes prediction accuracy (number of correct predictions divided by the total number of predictions).

ii. **Implement** the `confusion_matrix` function that manually constructs a confusion matrix. Each row represents the true class and each column represents the predicted class.

3.2.3 Problem 3

Perform the following tasks.

iii. For each mini-batch size, use your `kfold_split` function to generate training and test indices for each of the K folds.

iii. For each fold, initialize the weight matrix $\mathbf{W}$ (zeros) and train for a fixed number of epochs using mini-batch stochastic gradient descent.

iii. For each epoch, shuffle the training data with `np.random.permutation`, divide it into non-overlapping mini-batches, and sequentially update $\mathbf{W}$ on every mini-batch. Compute the gradient of the softmax cross-entropy loss manually (do not use built-in functions). At the end of each epoch, record the wall-clock time using `time.time()` and the accuracy using your implemented function.

Hyperparameters (fixed for all experiments):

- SGD learning rate: 0.1
- Epochs per fold: 500
- Mini-batch sizes: 1, 36, 72
- Number of folds (K): 10
- Weight initialization: zeros
- Regularization parameter: 0

3.2.4 Problem 4

Perform the following tasks.

iv. **Plot Fig. 15**: Training and testing accuracy versus epoch index for different mini-batch sizes. The plot shows the average training and test accuracies across epochs. Use solid lines for training accuracy and dashed lines for test accuracy.

iv. **Plot Fig. 16**: Training time versus epoch index for different mini-batch sizes. The plot shows the wall-clock time (in seconds) required for each epoch.

iv. **Plot Fig. 17** (set): Final average confusion matrices for each mini-batch size (one matrix per size). Each cell shows the number of instances for the predicted class (column) versus the true class (row).

3.2.5 Problem 5

For each figure (Figs. 15–17), discuss the results and relate them to the code implementation and the underlying theory.